Grids and Simulation

Topic 8 of 10

3 questions | skeleton and writing space only

Each question: the prompt, the function or class signature, and ruled lines to write your own solution.

1. Tic-Tac-Toe System with K Players and 3-in-a-Row Win

MEDIUM | asked 2026-05-18 | object-design, grid, simulation

Design a Tic-Tac-Toe system for k players on an n x n board. After each move, return the game status; any player wins immediately after forming 3 consecutive marks in a straight line. Input parameters: k players and an n x n board. Players take turns placing their marks on empty cells. A player wins as soon as they form 3 consecutive marks in a straight line, regardless of how large n is. After each move, return the correct game status, such as Game Over or <player> won.

```
class TicTacToe:

    def __init__(self, n, k, target=3):
```

```
    def inside(self, r, c):
```

```
    def run_length(self, r, c, dr, dc):
```

```
    def move(self, player, r, c):
```

2. Gate Entry / Exit Time Resolution with Tie-Break Rules

MEDIUM | asked 2026-05-04 | simulation, algorithm

Onsite coding round 2: given an array indexed by person id, each containing (timestamp, action in {enter, exit}), return per-person the actual time they passed through the gate after applying tie-break rules between simultaneous enters and exits.

```
def gate_pass_times(events, idle_first='exit'):
```

3. Babylon Tile-Stack Merge Game

HARD | asked 2025-12-29 | simulation, data-structure

Onsite NG round 1 (Munich office): implement a two-player tile-stack merge game ("Babylon") given the rules. Start state is 12 height-1 stacks of 4 colors (Y / W / G / B), 3 of each; a move places one stack on top of another under a constraint; the player who cannot move loses. 12 tiles initially, partitioned into 12 height-1 stacks. 4 colors: Y / W / G / B; exactly 3 tiles per color. Two players alternate moves.

```
def wins(stacks):
```